



From Bricks to Forms



DigiPass

Know Your Products!

Building Digital Product Passports with SHACL

A visual approach to structured data

No coding background required

Know Your Products!

COORDINATION AND KNOWLEDGE SHARING ACROSS MATERIALS DEVELOPMENT COMMUNITIES (CSA):
DigiPass CSA project has been funded by the European Commission for the programme HORIZON-CL4-2023-RESILIENCE-01, Grant Agreement No 101138510
WIKKI LIMITED, UK participant in Horizon Europe Project DigiPass, is supported by UKRI grant number 10100819: DigiPass



Heinz A. Preisig
Process Systems Engineering
NTNU, Trondheim
Norway



e-mail

Every Product has a Story



Pick any object in this room.

Where was it made? From what? By whom?

Is any material in it hazardous? Can it be repaired? Recycled?

The data exists — somewhere.

Spread across supplier spreadsheets, PDF certificates, email threads,
ERP systems that don't talk to each other.

Every Product has a Story



A Digital Product Passport (DPP) is a simple idea:

attach a structured, machine-readable data record to every product —
like a passport for things instead of people.

Scan a QR code → get the verified data record,
instantly, in any language, readable by any system.

The EU is making this mandatory across many sectors:
batteries, textiles, electronics, construction materials, machinery ...

The Data Exists. Problem -- Agreement



Supplier A reports: weight: 5 kg

Supplier B reports: mass: 5000 g

Supplier C reports: 5 ← unit unknown

All three mean the same thing. No computer can tell.

The problem is not missing data.

It is that everyone describes the same thing differently.

The Data Exists. Problem -- Agreement



For any DPP to work — regardless of product sector —
the data must be:

- ✓ Named consistently ("mass", not "weight" or "Gewicht")
- ✓ Typed consistently (number, not free text)
- ✓ Unitised consistently (kg, g, lb — from an agreed list)
- ✓ Validated automatically (catch errors before they propagate)
- ✓ Linked to shared meaning (so any system interprets it identically)

This is a data standards problem — and it has a W3C standard solution.



Digital Product Passports Are Hard

✗ TODAY

- Company A says: 'weight: 5kg'
- Company B says: 'mass: 5000g'
- Company C says: '5' (just a number!)

→ Computers can't understand this
→ Manual data entry & errors
→ Compliance headaches

✓ WITH SHACL

- Product.weight = 5
- Must be a number
- Must have a unit
- Unit can only be: kg, g, lb, oz

→ Machines can validate it
→ Forms guide data entry
→ Always consistent

SHACL: Rules that Computers can read ...



SHACL (Shapes Constraint Language) is a W3C standard for writing data rules that machines can automatically check and act on.

"mass" must be present

"mass" must be a decimal number

its unit must come from an agreed list (kg, g, lb, oz ...)

that list is linked to a shared ontology (QUDT)

SHACL: Rules that Computers can read ...



Write the rule once:

- ✓ any data claiming to be a DPP is validated against it
- ✓ a data-entry form is generated from it automatically
- ✓ any system — regardless of programming language or platform — can read, validate and exchange the data

SHACL: Rules that Computers can read



The key insight:

The form is not designed separately from the data model.
It is a direct consequence of the rule.

We use a battery DPP as our worked example throughout this talk.
The approach works for any product, any sector.



What If Data Had Grammar?

Like grammar rules in language, data needs rules too.

✗ "The product weighs about 5kg usually" (ambiguous)

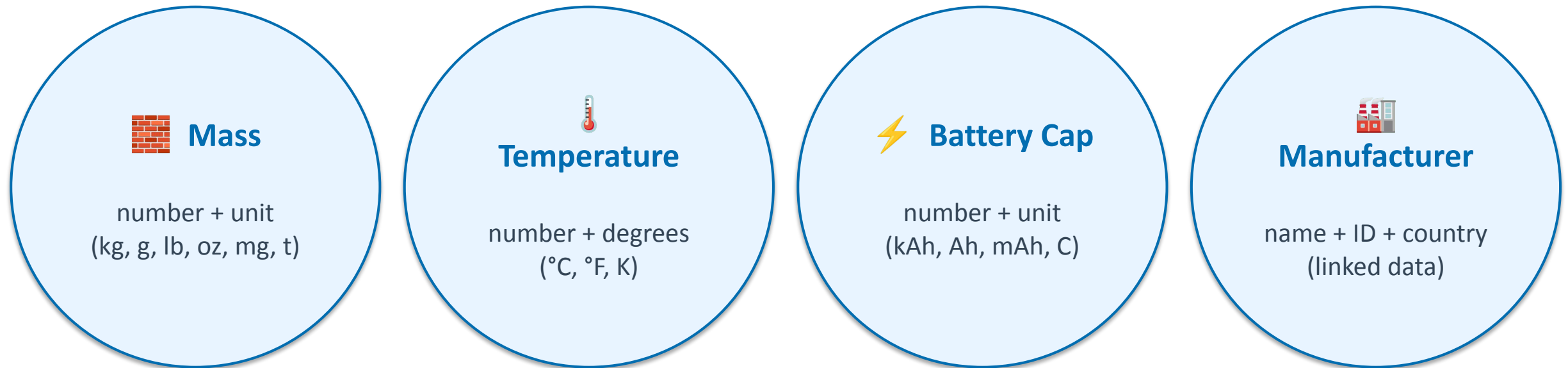
✓
Product.weight = 5
 type → number
 unit → kg (or g, lb, oz)
required → yes
checked → automatically

SHACL is the W3C standard for writing these rules.



Lego Bricks for Data Schemas

Instead of writing rules from scratch every time → create reusable bricks once, snap them together.



Define once → reuse across every product schema in your organisation.



Creating a Reusable Brick

Point-and-click — no coding needed:

Name	BatteryCapacity	
Path	ex:batteryCapacity	
Datatype	xsd:decimal	
Class ✦✦	qudt:ElectricCharge	← triggers magic

Result →

Unit dropdown appears automatically:

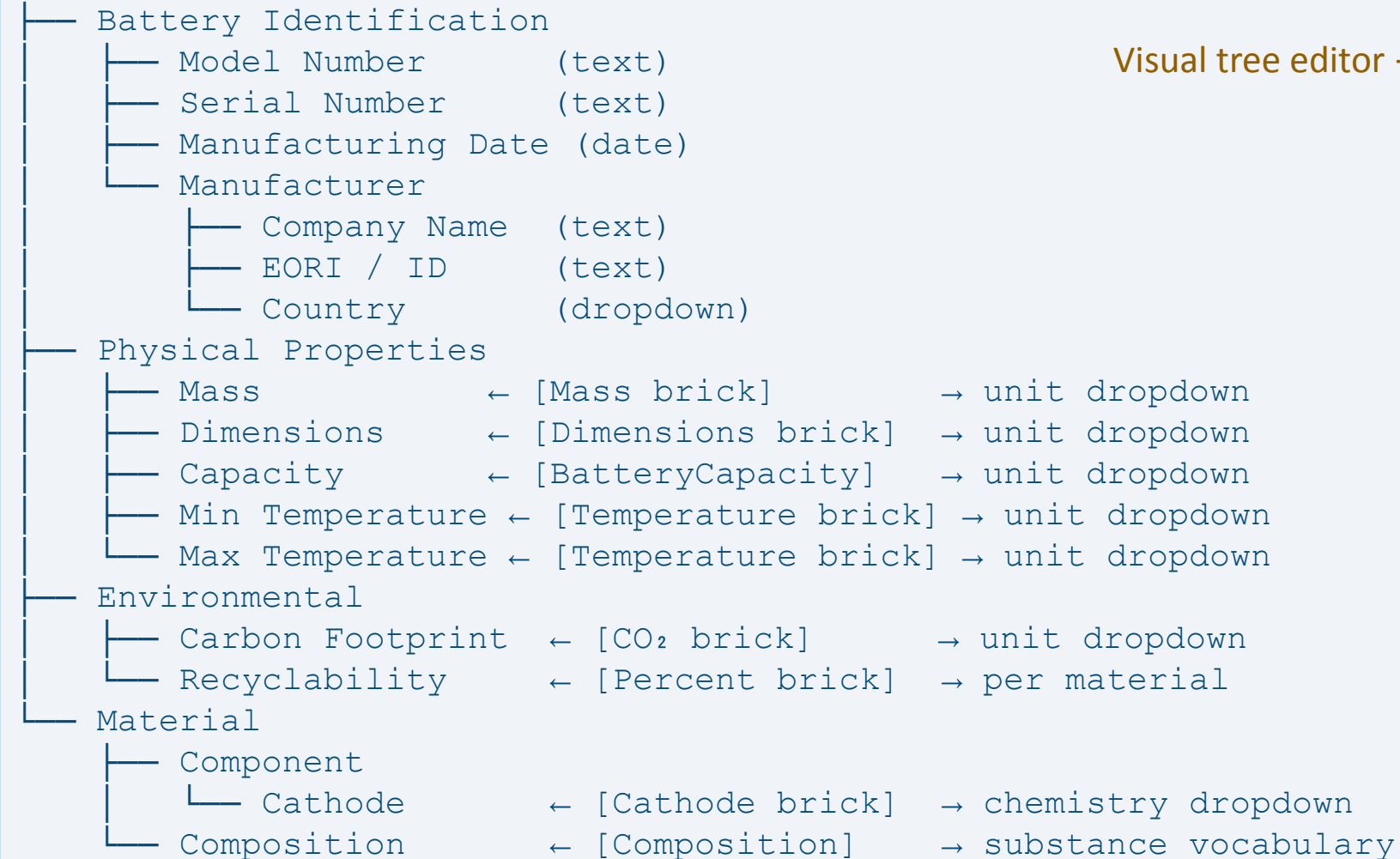
- ▼ Ampere-hour (Ah)
- Milliampere-hour (mAh)
- Coulomb (C)
- Kiloampere-hour (kAh)

The system looks up applicable units from the QUDT ontology — no manual list needed.

The Schema Editor -- Assemble Schema



EV Battery Schema



Visual tree editor — drag bricks in, set connections, done.

One Source → Many Outputs



The Magic: One Brick, Three Things

Brick (JSON)

```
{
  name: 'Mass',
  datatype: xsd:decimal,
  class: qudt:Mass,
  units: [kg,g,lb,...]
}
```

SHACL Rules

```
sh:property [
  sh:path ex:mass ;
  sh:datatype xsd:decimal ;
  sh:class qudt:Mass ;
  sh:in (kg g lb oz) ;
] .
```

User Form

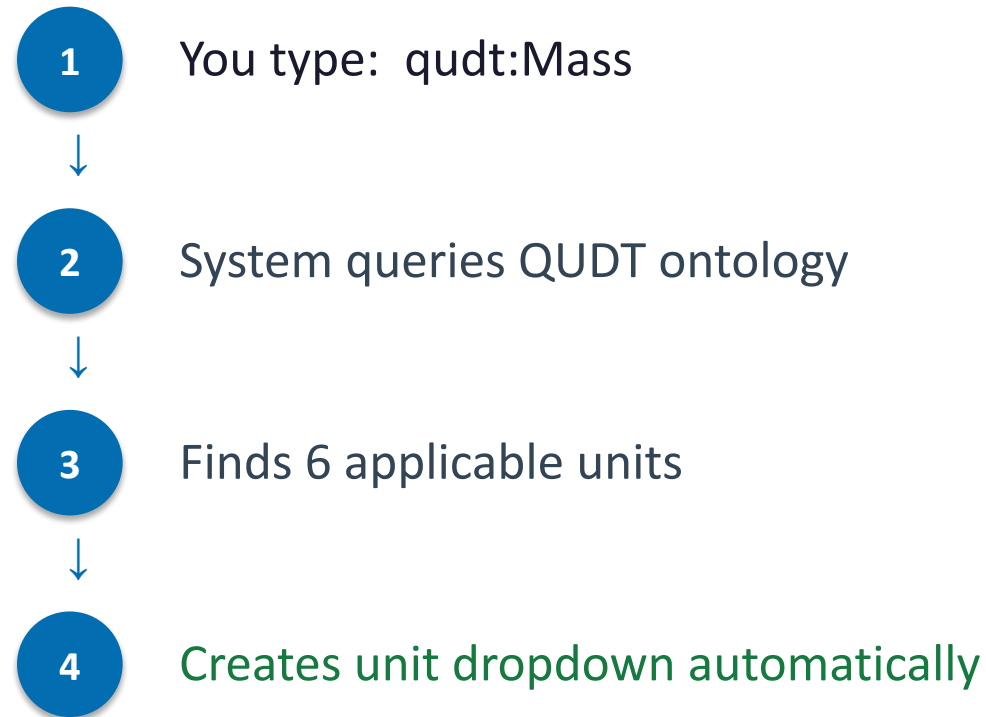
Mass

5	▼ kg
---	------

Change brick once
→ all forms update



Smart Widgets from Context



Works for any physical quantity: temperature, pressure, energy, voltage ...



How It All Connects

Shared Ontologies QUDT · Schema.org · FOAF · SKOS (17 vocabularies)

Brick Library

Reusable building blocks (JSON)

Schema Composer

Assemble bricks into product blueprints

SHACL Exporter

Generates validation rules (Turtle)

Form Generator

HTML interface the data-entry clerk sees

Validated Data

Ready for Digital Product Passports



Why This Matters

	Before	After
Schema definition time	3 months	1 week
Unit configuration	manual lookup	automatic
Validation errors	50+ (found late)	2 (caught early)
Consistency	varies per team	standardised
Non-technical authors	impossible	point-and-click

Key Takaways



Bricks

Reusable, semantic data components — define once, use everywhere



SHACL

Standardised validation rules — machines can check your data



Enrichment

Ontology-driven smart widgets — unit dropdowns appear automatically



Forms

Automatic user interfaces — no coding needed for data entry



Reuse

Change a brick once → every form using it updates

Getting Started



One Download, One Command

1 Download

`docker pull heinzpreisig/dash-gui` or clone the GitHub repository

2 Run

`docker-compose up` (or `uv run python run_brick_app_web.py`)

3 Open

Browser → `http://localhost:5001` for the Brick Editor

4 Your data

Create bricks, assemble schemas, export SHACL forms

Questions?



Q: Can non-programmers use this?

A: Yes — brick_app and schema_app are point-and-click web tools.

Q: What about our own vocabulary?

A: Load any RDF/OWL ontology — enrichment works with it automatically.

Q: Does it connect to our database?

A: Exported SHACL validates data; can export to JSON/XML/RDF for any system.



Know Your Products!



COORDINATION AND KNOWLEDGE SHARING ACROSS MATERIALS DEVELOPMENT COMMUNITIES (CSA):

DigiPass CSA project has been funded by the European Commission for the programme HORIZON-CL4-2023-RESILIENCE-01, Grant Agreement No 101138510
WIKKI LIMITED, UK participant in Horizon Europe Project DigiPass, is supported by UKRI grant number 10100819: DigiPass

